

PROGRAM 2

Develop a multistage Docker-file for container orchestration

Step 1 : Create the following files inside a folder (src/index.js, DockerFile)

```
GNU nano 7.2
FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

RUN npm run build

FROM node:20-alpine

WORKDIR /app

COPY --from=builder /app/package*.json ./
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules

EXPOSE 3000

CMD ["node","dist/index.js"]
```

```
GNU nano 7.2
const express = require("express");
const app=express();
const port=3000;

app.get("/", (req, res) => {
  res.send("Hello from Multi-stage Docker");
});

app.listen(port, () => {
  console.log(`Server listening on port ${port}`);
});
```

Step 2 : For having the package.json and package-lock.json we have to use the commads:

npm init -y ; npm install express (node_modules)

Step 3 : Open that in a terminal. Run these commands to build and run Docker

```
1rv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/Downloads/p2$ nano Dockerfile
1rv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/Downloads/p2$ cd src
1rv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/Downloads/p2/src$
1rv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/Downloads/p2/src$
1rv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/Downloads/p2/src$ nano index.js
1rv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/Downloads/p2/src$ cd ..
1rv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/Downloads/p2$ docker build -t p2 .
[+] Building 4.5s (14/14) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 363B                                0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine  3.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 44.12kB                                  0.0s
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:6178e78b972f79c335df281f4b7674a2d85071aae2af020ffa39f0a770265435  0.0s
=> CACHED [builder 2/6] WORKDIR /app                             0.0s
=> CACHED [builder 3/6] COPY package*.json ./                     0.0s
=> CACHED [builder 4/6] RUN npm install                           0.0s
=> [builder 5/6] COPY . .                                         0.8s
=> [builder 6/6] RUN npm run build                                0.4s
=> CACHED [stage-1 3/5] COPY --from=builder /app/package*.json ./ 0.0s
=> CACHED [stage-1 4/5] COPY --from=builder /app/dist ./dist     0.0s
=> CACHED [stage-1 5/5] COPY --from=builder /app/node_modules ./node_modules 0.0s
=> exporting to image                                             0.0s
=> => exporting layers                                             0.0s
=> => writing image sha256:1ab8195f5acec085408b6ce53f102faa52b74346e78aca333513066ae729a9f0 0.0s
=> => naming to docker.io/library/p2                              0.0s
1rv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/Downloads/p2$ docker run -t -p 3000:3000 p2
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint frosty_kowalevski
(740034bb426daef3dad14178a20df5ec860b2f9352ba761ec8ea694a93c3001): Bind for 0.0.0.0:3000 failed: port is already allocated

Run 'docker run --help' for more information
1rv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/Downloads/p2$ docker run -t -p 8000:3000 p2
Server listening on port 3000
█
```

Step 3 : The output of the program

